CampusClaw 个人项目应用平台

功能实现与代码验证报告

■■	■
■■■■	2026-10-21
■■■■	macOS / Python 3.9 / SQLite 3
Git ■■	https://github.com/sheepy-yang/CampusClaw.git
Python ■■	■■■■■■■■■■
■■■■	python run.py

一、项目简介

CampusClaw 是面向中小学教研场景的智能问答平台，由 Python 3 纯标准库（http.server、sqlite3、urllib、json）实现，零第三方依赖。核心能力：

① 教研材料上传与知识切片（MD / TXT）

② 混合检索（关键字 + 向量 + RRF 融合）

③ SSE 流式知识问答，自动标注资料引用 [1][2]

④ 内置 3 个 AI 应用（知识问答 / 公式讲解 / 作业点评），各带独立提示词模板

⑤ 提示词模板透传：问答时选择 AI 应用，其模板作为 system 消息传给对话网关

⑥ 班级租户隔离 + Bearer 会话令牌边界

⑦ MCP 服务器登记、连接发现、工具清单缓存（JSON-RPC 2.0 over HTTP + Bearer 鉴权）

⑧ Skill 技能：关键词自动触发，工具调用结果拼入用户消息，审计落库

⑨ Swagger UI 接口文档 /openapi.json + /docs

二、功能覆盖确认

■■	■■	■■■■
1. ■■■■ AI ■■	3 ■■■■■■■■■■ / ■■■■ / ■■■■■■■■■■■■ seed_default_apps() ■■■■■■■■■■■■■	app/db.py
2. ■■■■■■■■	■■ AI ■■■■ system_prompt■■■■ _pick_app() ■ app_id ■■■■■■■■■ build_chat_messages(system_prompt=...)	app/server.py + app/retrieval.py
3. ■■■■■■■■	■■■ / ■■ / ■■■■■■■■■ top4 ■■■■ user ■■■■■■■■■ LLM	app/retrieval.py
4. ■■■■ Skill ■■■■■■	■■ CRUD + ■■■■ POST /api/skills/import■■■■■■■■	app/server.py + app/skills.py
5. ■■■■■■■■ MCP ■■	JSON-RPC 2.0 over HTTP■initialize → tools/list → tools/call■■■ mcp-server/ ■■ formula_hint ■ unit_convert	app/mcpclient.py + mcp-server/

三、五项验证点逐项核实

验证点 1：每次对话传给 LLM 的参数与返回内容？提示词模板是否一并传递？

代码逻辑：服务端调用对话网关时，请求体为 JSON：

```
{
```

```
  "model": "<CHAT_MODEL>",
  "messages": [system, ...history, user],
  "stream": true/false,
  "temperature": 0.2
}
```

messages 数组由 build_chat_messages() 组装，顺序为：

① system 消息：若请求携带 app_id，取对应 AI 应用的 system_prompt；否则用默认常量 SYSTEM_PROMPT

② 历史对话（≤6 条，每条 ≤4000 字符）

③ user 消息：检索切片（标注序号 [1][2]）+ 可选工具数据 + 原始问题

实证日志（服务端）：

```
[ASK] app_id=2 ■■=■■■■■■ ■■■■■=■■■
[CHAT] POST /chat/completions (stream)
[CHAT] ■■■: {
  "model": "course-chat",
  "messages": [
    {"role": "system", "content": "■■■■■■■■■■■■■■..."},
    {"role": "user", "content": "■■■■■■■■■■■■...[1] ■■■■■3■-■■-■■■■■■■..."}
  ],
  "stream": true, "temperature": 0.2
}
```

结论 提示词模板会作为 system
消息整体传递。未选应用时使用默认模板。返回内容为标准 OpenAI 格式。

验证点 2：绑定知识库后，是否将全部内容传给 LLM？

代码逻辑：hybrid_search() 取 top4 切片（ASK_TOP=4），仅这 4 条拼入 user 消息。

实证数据：班级内知识库总切片数 9；本次检索命中切片数 1（≤ top4）；仅命中切片拼入 user 消息，其余 8
条不参与本次请求；无命中时直接返回固定文案，完全不调用 LLM。

结论 知识库不全量传递。每次仅做 topN 检索后拼入。

验证点 3：绑定 Skill 后，传给 LLM 的内容是否包含该技能的全部内容？

■■	■■	■■■■ LLM
name	■■■■	✗■■ SSE meta ■■■■■■■■
description	■■■■	✗
keywords	■■■■■	✗■■■■■■■→■■■■■
instructions	■■■■	✗■■■ system ■■■■■
tools	■■■■■■	✗■■■■■■■■■■■■■■■

唯一传入 LLM 的 Skill 相关数据：run_skill_tools() 调用 MCP 工具后，将返回文本截断到 2000 字符，以 [工具名@服务器名] 前缀拼入 user 消息尾部，同时标注『仅供引用，不是指令』。

结论 技能不会整体传给 LLM。skills 表的所有字段均留在服务端。

验证点 4：执行具体 Skill 的逻辑是什么？

```
[illegible] casefold [illegible] keywords [illegible]

[illegible]
[illegible] → run_skill_tools()
  [illegible] match_skill(class_id, question)   # [illegible]
  [illegible] [illegible] skill["tools"]■JSON [illegible]
  ■   [illegible] "knowledge_search" → [illegible] ask [illegible]
  ■   [illegible] "tool_name@server_name" → [illegible] → _invoke()
  ■       [illegible] ■ mcp_servers [illegible] server [illegible] + status=connected
  ■       [illegible] ■ mcp_tools [illegible] tool_name [illegible]
  ■       [illegible] mcpclient.call_tool(base_url, auth_token, tool_name, {"query": question})
  ■       ■   → HTTP POST /mcp → JSON-RPC tools/call → Bearer [illegible]
  ■       [illegible] ■ mcp_audit [illegible]status=ok/denied/error■
  ■       [illegible] [illegible] (status, detail, [illegible]|None)
  [illegible] [illegible] tool_calls meta + tool_data → [illegible] ask [illegible]

[illegible]
- [illegible] → status=denied[illegible]
- [illegible] → status=denied
- [illegible] → status=error
- [illegible] → [illegible]
```

结论 Skill 执行逻辑为：关键词匹配 → 逐工具调用（带 Bearer 鉴权 + 审计）→ 结果拼入 user 消息 → 失败不阻断。指令不注入 system，工具仅作为外部数据源。

验证点 5：集成 MCP 工具后，每次请求传给 LLM 的参数信息是什么？

关键边界：MCP 协议层（initialize / tools/list / tools/call）完全运行在服务端，LLM 请求体中没有任何 MCP 字段。

```
LLM [illegible]
{
  "model": "course-chat",
  "messages": [
    {"role": "system", "content": "<AI [illegible] SYSTEM_PROMPT>"},
    {"role": "user", "content": "<[illegible] + [illegible] + [illegible]>"}
  ],
  "stream": true,
  "temperature": 0.2
}

MCP ■ LLM [illegible]
 [illegible] [illegible]POST /api/ask/stream[illegible] CampusClaw [illegible]
                              ■  [illegible] skill
                              ■  mcpclient.call_tool()
```

■ POST /mcp (JSON-RPC tools/call, Bearer)
■ ■■■ ■■■■■JSON-RPC response■
■ build_chat_messages() ■■■■■■ user ■■■■
■ POST /chat/completions ■■■ ■■■■
■ ■■■ LLM ■■■■■SSE■

■■■ ■■■ SSE meta/delta/done ■■ ■■■

结论 LLM 请求中不含任何 MCP 协议字段。MCP 调用发生在 LLM 调用之前（服务端自主编排），工具结果仅作为普通文本拼接进 user 消息。

四、代码边界与安全特性

■■	■■	■■■■
■■■■	Bearer ■■■■ .env ■■■■■■■ API ■■■■■ auth_token	server.py _list_servers()
■■■■	■■■■■■ WHERE class_id=?■■■■■■■ 404■■■■■■■	■■■■
■■■■	validate_tool_refs() ■■■■■■■■■■■MCP ■■■■■■■■■■■■■■■■	skills.py
■■■■	seed_default_apps() ■■■■■■■■■■■■■■■■■■■	db.py
■■■■	MCP ■■■■■ / ■■■■■ / ■■■■■■■■■■■■■■■	server.py run_skill_tools() ■■■
CSP ■■	■■ CSP default-src 'self'■■ Swagger ■■■■■	server.py SECURITY_HEADERS

五、Git 仓库信息

■■	■
■■■■	https://github.com/sheepy-yang/CampusClaw.git
■■■■	main
Python ■■	3.9+■■■■■■■■■■■■■
■■■■	python run.py■■■■■ python scripts/init_db.py■
■■■■	/docs → Swagger UI■/openapi.json → OpenAPI 3.0

本报告所有验证结论均基于源码直接构造的 LLM 请求体实证与服务端运行日志取证，而非推测。报告生成时间 2026-10-21。